

Java2 Day04 핵심 정리

리플렉션과 변경에 유리한 코드 작성 방식을 쉬운 말과 코드로 정리

주제: 리플렉션과 변경에 유리한 코드

대상 독자: 인터페이스와 객체 생성 이후 리플렉션을 처음 배우는 Java 초보자

작성 기준 자료: Day4 TXT, Day4 원본 PDF, java2prjDay4 day4 소스 코드

생성일: 2026-05-21

Class 클래스

실행 중 클래스 정보를 담는 객체입니다.

Reflection

실행 중 클래스, 필드, 메서드를 읽고 사용할 수 있는 기술입니다.

동적 객체 생성

문자열 클래스명으로 객체를 만드는 방식입니다.

느슨한 결합

구체 클래스가 바뀌어도 기존 코드 수정이 적은 구조입니다.

0. 목차와 학습 로드맵

단원	학습 내용
1	Day04 전체 개요
2~7	리플렉션, Class 객체, 필드/메서드 정보, 동적 객체 생성
8~10	변경에 유리한 코드, 느슨한 결합, Properties 설정
11	2인 1조 실습 방향
12~14	연습 문제, 확인 불가 항목, 최종 요약

1. 먼저 class 객체가 "클래스 정보"를 담는다는 점을 이해합니다.
2. 필드와 메서드 정보를 실행 중에 읽는 흐름을 봅니다.
3. invoke로 메서드를 호출하는 구조를 확인합니다.
4. 설정 파일과 리플렉션을 이용해 구현체를 바꾸는 방법을 이해합니다.
5. 느슨한 결합이 왜 변경에 유리한지 정리합니다.

원본 자료	확인 내용	반영 방식
4일차.txt	전체 내용 확인	리플렉션, 변경에 유리한 코드 학습 목표 반영
스트림문제.txt	전체 내용 확인	Day4 보조 실습으로 확인
Day4 원본 PDF 2개	파일 존재, PDF 헤더, 페이지 구조 확인	리플렉션과 변경에 유리한 코드 주제 반영
workspace/java2prjDay4/src/day4	48개 파일 구조와 대표 예제 확인	Member, Animal, Calculator, Properties 코드 예제로 반영

확인 제한: PDF 본문 세부 텍스트는 완전 추출하지 못했습니다. TXT 전체 내용과 실제 소스 코드 확인 내용을 중심으로 재구성했습니다.

1. Day04에서 배우는 Java 문법

Day04는 실행 중에 클래스 정보를 다루는 **리플렉션(reflection)**과 코드를 더 바꾸기 쉽게 만드는 **느슨한 결합**을 배웁니다. Day01의 인터페이스 개념이 Day04에서 설정 파일, 리플렉션, 의존성 주입과 연결됩니다.

개념	쉬운 뜻	대표 코드
Class 객체	클래스 정보를 담는 객체	<code>Class.forName("패키지.클래스")</code>
Reflection	실행 중 클래스 정보를 보고 사용하는 기술	<code>getDeclaredMethods()</code>
Method invoke	메서드를 이름으로 찾아 실행	<code>method.invoke(obj)</code>
Properties	외부 설정 파일을 key-value로 읽음	<code>p.getProperty("animal")</code>
느슨한 결합	구체 클래스가 아니라 인터페이스에 의존	<code>Calculator calculator;</code>

핵심: Day04의 목표는 "코드에 클래스 이름을 직접 박아 넣지 않고도 실행 시점에 필요한 객체를 선택하는 구조"를 이해하는 것입니다.

2. 리플렉션이란?

리플렉션은 실행 중에 클래스 정보를 거울처럼 들여다보는 기술입니다. 클래스 이름, 필드, 메서드, 생성자 정보를 읽을 수 있고, 객체를 만들거나 메서드를 호출할 수도 있습니다.

```
Class clazz = Class.forName("day4.reflect.Member");
```

위 코드는 문자열로 적힌 클래스 이름을 이용해서 Member 클래스 정보를 가져옵니다. 여기서 중요한 점은 **컴파일할 때가 아니라 실행 중에 클래스 정보를 읽는다는 것**입니다.

3. 클래스 정보를 얻는 방법

방법	코드	특징
클래스명.class	<code>Class clazz = Member.class;</code>	컴파일 시점에 클래스를 알고 있음
객체.getClass()	<code>Class clazz = member.getClass();</code>	이미 만들어진 객체에서 얻음
Class.forName()	<code>Class clazz = Class.forName("day4.reflect.Member");</code>	문자열 클래스명으로 실행 중 로딩

4. 예제 클래스 Member

```

public class Member {
    String id;
    String pw;

    public Member() {
        this("기본아이디", "기본비번");
    }

    public Member(String id, String pw) {
        this.id = id;
        this.pw = pw;
    }

    public void printInfo() {
        System.out.println("id=" + id);
        System.out.println("pw=" + pw);
    }

    public void printSquare(int su) {
        System.out.println(su * su);
    }
}

```

이 클래스는 리플렉션 실습용입니다. 필드 정보, 메서드 정보, 객체 생성, 메서드 호출을 모두 연습할 수 있습니다.

5. 필드와 메서드 정보 얻기

```
Class clazz = Class.forName("day4.reflect.Member");

Field[] fields = clazz.getDeclaredFields();
for (Field field : fields) {
    System.out.println(field);
}

Method[] methods = clazz.getDeclaredMethods();
for (Method method : methods) {
    System.out.println(method.getName());
}
```

메서드	뜻
getDeclaredFields()	클래스에 선언된 필드 목록을 가져옴
getDeclaredMethods()	클래스에 선언된 메서드 목록을 가져옴
getDeclaredMethod("이름", 타입...)	특정 메서드 하나를 가져옴

6. 리플렉션으로 객체 만들기

```
Class clazz = Member.class;
Member member = (Member) clazz.newInstance();
member.printSquare(3);
```

newInstance()는 기본 생성자를 사용해서 객체를 만듭니다. 최근 Java에서는
getDeclaredConstructor().newInstance() 사용을 더 권장하지만, 수업 예제에서는 리플렉션 개념 이해를 위해
newInstance()를 사용합니다.

```
Class clazz = Class.forName("day4.reflect.Member");
Object obj = clazz.getDeclaredConstructor().newInstance();
```

일반 객체 생성: new Member()

리플렉션 객체 생성: 클래스 정보를 얻은 뒤 실행 중 객체 생성

7. 리플렉션으로 메서드 호출하기

```
Member m = new Member();
Class clazz = Class.forName("day4.reflect.Member");

Method method = clazz.getDeclaredMethod("printInfo");
method.invoke(m);

Method method2 = clazz.getDeclaredMethod("printSquare", int.class);
method2.invoke(m, 3);
```

invoke는 메서드를 실행합니다. 첫 번째 인수에는 메서드를 실행할 객체가 들어가고, 두 번째 이후 인수에는 메서드에 전달할 값이 들어갑니다.

주의: 리플렉션은 강력하지만 오타나 타입 오류가 실행 중에 터질 수 있습니다. 그래서 일반 코드보다 더 신중하게 써야 합니다.

8. 변경에 유리한 코드

변경에 유리한 코드는 특정 클래스에 강하게 묶이지 않는 코드입니다. 예를 들어 `new Dog()`를 코드 곳곳에 직접 쓰면 `Cat`으로 바꿀 때 여러 곳을 수정해야 합니다. 대신 설정 파일과 리플렉션을 사용하면 외부 파일만 바뀌도 구현체를 바꿀 수 있습니다.

```
public class Animal {
    public void bark() {
        System.out.println("동물이 짖는다");
    }
}

public class Dog extends Animal {
    @Override
    public void bark() {
        System.out.println("멍멍");
    }
}

public class Cat extends Animal {
    @Override
    public void bark() {
        System.out.println("야옹");
    }
}
```

```
// config.txt
animal=day4.변경에유리한코드.Dog
```

```
Properties p = new Properties();
p.load(new FileReader("src/day4/변경에유리한코드/config.txt"));

String className = p.getProperty("animal");
Class clazz = Class.forName(className);
Animal animal = (Animal) clazz.newInstance();

animal.bark();
```

이제 Dog 대신 Cat을 쓰고 싶으면 Java 코드를 고치는 대신 설정 파일의 클래스명만 바꾸면 됩니다.

9. 느슨한 결합: 계산기 예제

계산기 예제는 Day04의 핵심 구조를 잘 보여줍니다. 프로그램이 특정 계산기 클래스에 직접 의존하지 않고, Calculator 인터페이스에 의존합니다.

```
public interface Calculator {
    int add(int su1, int su2);
}
```

```
public class C implements Calculator {
    @Override
    public int add(int su1, int su2) {
        System.out.println("C add");
        return su1 + su2;
    }
}
```

```
public class D implements Calculator {
    @Override
    public int add(int su1, int su2) {
        System.out.println("D add");
        return su1 + su2;
    }
}
```

```
public class Program {
    Calculator calculator;

    public void setCalculator(Calculator calculator) {
        this.calculator = calculator;
    }

    public void run() {
        int total = calculator.add(10, 20);
        System.out.println(total);
    }
}
```

프로그램은 c인지 d인지 몰라도 됩니다. Calculator 인터페이스의 add만 호출하면 됩니다.

구조	문제점/장점
new C()를 프로그램 안에 직접 작성	C에서 D로 바꾸려면 Java 코드 수정 필요
Calculator 인터페이스에 의존	C, D, SMCalculator 등 구현체 교체가 쉬움
설정 파일 + 리플렉션	코드 수정 없이 외부 파일로 구현체 선택 가능

10. Properties 파일 읽기

```
Properties properties = new Properties();
properties.load(new FileReader("src/day4/계산기_느슨한결합/config.txt"));

String calculatorClass = properties.getProperty("calculator");
```

Properties는 key=value 형식의 설정 파일을 읽는 데 사용합니다. 설정 값을 코드 밖으로 빼면 프로그램 변경에 더 유리합니다.

11. 2인 1조 실습 방향

- 원하는 클래스를 만들고 Class.forName으로 클래스 정보를 가져오기
- getDeclaredFields, getDeclaredMethods로 필드와 메서드 정보 출력하기
- getDeclaredMethod와 invoke로 메서드 호출하기
- 인터페이스를 만들고, 구현 클래스 여러 개를 만들기
- 설정 파일에 사용할 구현 클래스 이름을 적고, 리플렉션으로 객체 생성하기
- 요리사 프로그램이라면 Store는 Cook 인터페이스만 알고, 실제 요리사는 설정 파일로 바꾸기

12. 연습 문제와 정답 해설

문제

1. Class.forName()은 무엇을 하는가?
2. 리플렉션으로 메서드를 실행할 때 사용하는 메서드는 무엇인가?
3. 설정 파일과 리플렉션을 함께 쓰면 어떤 장점이 있는가?
4. 느슨한 결합에서 프로그램은 구체 클래스와 인터페이스 중 무엇에 의존해야 하는가?

정답 및 해설

번호	정답 해설
1	문자열로 받은 클래스 이름을 이용해 실행 중 클래스 정보를 로딩합니다.
2	Method.invoke()를 사용합니다.
3	Java 코드를 직접 수정하지 않고 설정 파일의 클래스명 변경으로 구현체를 교체할 수 있습니다.
4	구체 클래스가 아니라 인터페이스에 의존해야 변경에 유리합니다.

13. 원본 출처와 확인 불가 항목

항목	이유	필요한 추가 자료
Day4 원본 PDF 세부 본문 전체 추출	PDF 텍스트 추출 도구 없음	텍스트 추출본 또는 OCR 결과
코드 실행 결과	이번 자료 제작에서는 실행 검증하지 않음	Eclipse 또는 javac 실행 환경 확인

14. 마지막 핵심

주제	반드시 기억할 말
리플렉션	실행 중 클래스 정보를 읽고 객체 생성, 메서드 호출까지 할 수 있는 기술 이다.
Class.forName	문자열 클래스명으로 클래스 정보를 로딩한다.
Method.invoke	메서드 정보를 이용해 특정 객체의 메서드를 실행한다.
Properties	외부 설정 파일을 읽어서 프로그램 동작을 바꿀 수 있게 한다.
느슨한 결합	구체 클래스가 아니라 인터페이스와 설정에 의존해서 변경 포인트를 줄인다.